

RECETAS ASTURIANAS

Documento de Arquitectura y Diseño

Asignatura: Informática Móvil
Universidad de Oviedo
Escuela Politécnica de Ingeniería de Gijón
Curso académico 2025/2026 · Mayo 2026

Alumno: Jose Soeiro Fuillerat

1. Introducción

Recetas Asturianas es una aplicación Android nativa desarrollada como trabajo individual de la asignatura Informática Móvil de la Universidad de Oviedo. Su objetivo es permitir al usuario explorar, buscar, filtrar y guardar recetas de la gastronomía asturiana, obtenidas de una API REST pública del Principado de Asturias.

1.1 Información del Proyecto

Campo	Valor
Nombre	Recetas Asturianas
Package	es.uniovi.recetasasturianas
Asignatura	Informática Móvil — Universidad de Oviedo
Lenguaje	Kotlin 1.9.22
SDK mínimo	26 (Android 8.0 Oreo)
SDK objetivo	35 (Android 15)
Arquitectura	MVVM (Model-View-ViewModel)
Fuente datos	API pública Turismo Asturias Profesional + fallback assets

1.2 Requisitos Contractuales

La aplicación cumple las siguientes características:

- Fuente de datos: catálogo abierto del Principado de Asturias.
- Listado con nombre, imagen y restaurante/autor.
- Vista de detalle: ingredientes, preparación, tiempo y trucos.
- Enlace al restaurante embebido (WebView, no navegador externo).
- Búsqueda por nombre, restaurante e ingredientes.
- Filtrado por tiempo de preparación (rápida / media / larga).
- Guardado de favoritos con persistencia en Room.
- Recordar preferencias de usuario (orden, filtros activos).
- Enlace al restaurante embebido dentro de la app (condición extra del profesor por ausencia de datos GPS).

2. Arquitectura

La aplicación sigue el patrón MVVM recomendado por Google, con separación clara de responsabilidades en tres capas y un Repository que actúa como única fuente de verdad.

2.1 Capas de la Aplicación

Capa de Datos (Data Layer)

Gestiona todas las fuentes de datos de forma transparente para las capas superiores:

- API remota mediante Retrofit + Moshi (deserialización JSON).
- Base de datos local SQLite a través de Room (recetas + favoritos).
- Archivo estático de fallback en assets/recetas.json (~276 KB).
- RecipeRepository: fuente única de datos con lógica de fallback automático.

Capa de Presentación (UI Layer)

Fragmentos y ViewModels que implementan las pantallas de la aplicación:

- ViewModel + LiveData para comunicación reactiva con la UI.
- Navigation Component para la navegación entre fragmentos.
- Material Design 3 como sistema de diseño.
- ViewBinding para el acceso tipado a las vistas.

Capa de Utilidades

- NetworkUtils: detección de conectividad para gestionar el modo offline.

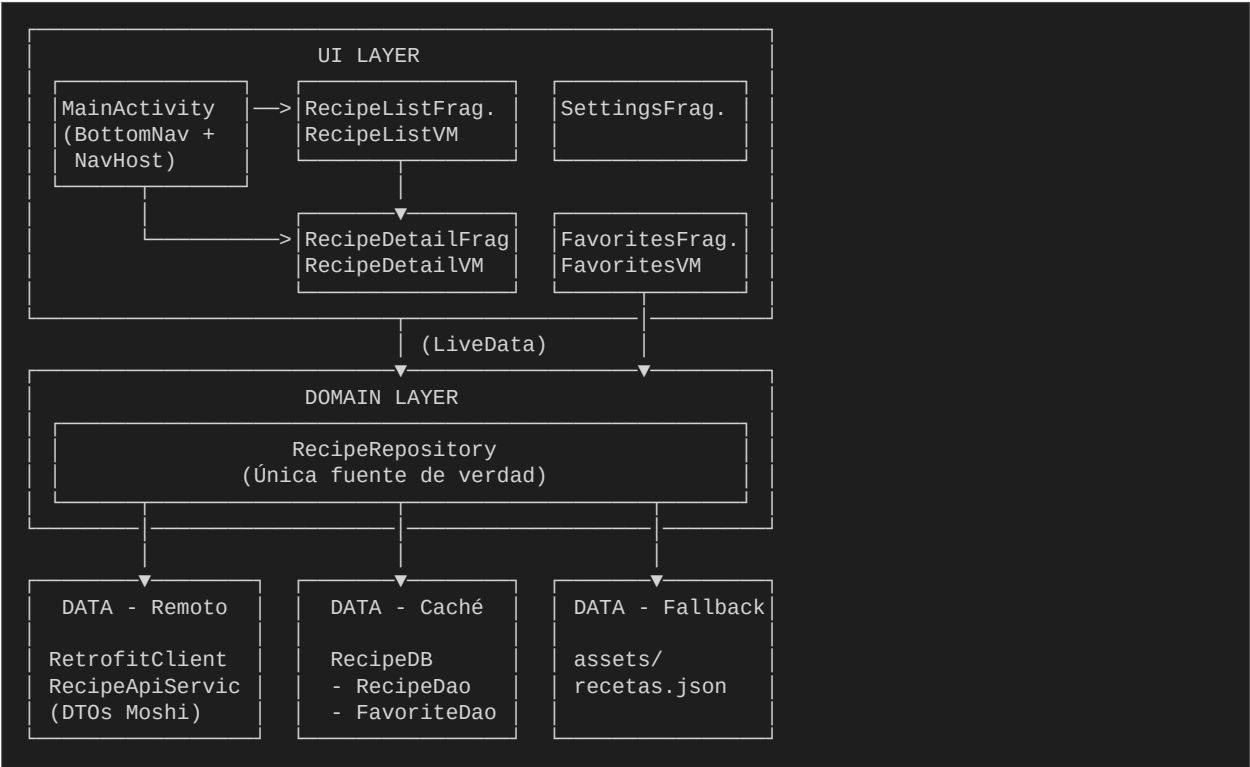
2.2 Flujo de Datos

El flujo de datos sigue este orden de prioridad:

1. RecipeListViewModel solicita datos a RecipeRepository.
2. RecipeRepository intenta la API remota (Retrofit + Moshi).
3. Si la API falla, carga desde assets/recetas.json (Moshi).
4. Los datos se cachean en Room (RecipeDao / FavoriteDao).
5. El ViewModel expone LiveData que el Fragment observa.
6. La UI se actualiza reactivamente ante cualquier cambio.

2.3 Esquema de Clases Principales

El siguiente diagrama muestra la relación entre los componentes clave de la arquitectura:



2.4 Endpoints y Fuentes de Datos

Tipo	URL	Estado
Principal (VPN universidad)	http://156.35.163.145/json/RecetasCocina.json	Activo por defecto
Alternativa pública	https://www.turismoasturiasprofesional.es/open-data/	Configurado, inactivo
Fallback local	app/src/main/assets/recetas.json	Activado si API falla

2.5 Base de Datos Room

Tabla recipes

Campo	Tipo	Descripción
id	Int	ID único (clave primaria)
name	String	Nombre de la receta
restaurant	String	Nombre del restaurante

timeMinutes	Int?	Tiempo en minutos (nullable)
ingredientsHtml	String	Ingredientes en HTML
preparationHtml	String	Preparación en HTML
tipsHtml	String?	Trucos en HTML (nullable)
notesHtml	String?	Notas en HTML (nullable)
imageUrl	String?	URL de la imagen (nullable)
restaurantUrl	String?	URL del restaurante (nullable)
cachedAt	Long	Timestamp de caché (epoch millis)

Tabla favorites

Campo	Tipo	Descripción
recipeId	Int	FK a recipes.id (clave primaria)
savedAt	Long	Timestamp de cuando se guardó

3. Estructura de Carpetas

El proyecto sigue una organización por paquetes coherente con la arquitectura MVVM:

```
app/src/main/java/es/uniovi/recetasasturianas/
├── App.kt                # Application class
├── SplashActivity.kt     # Splash Screen (Android 12+)
├── MainActivity.kt       # Activity principal con Navigation
├── data/
│   ├── local/           # Persistencia local (Room)
│   ├── model/           # Entidades Room
│   ├── remote/          # Cliente de red (Retrofit + Moshi)
│   │   ├── dto/         # DTOs + Mappers
│   │   └── repository/  # Fuente única de datos
├── ui/
│   ├── list/            # Lista de recetas
│   ├── detail/          # Detalle de receta
│   ├── favorites/       # Lista de favoritos
│   ├── settings/        # Preferencias de usuario
│   └── webview/         # WebView embebido
└── util/
    └── NetworkUtils.kt  # Detección de conectividad
```

3.1 Recursos (app/src/main/res/)

Carpeta	Contenido
layout/	Layouts de Activity y Fragments (teléfono)
layout-sw600dp/	Layouts adaptativos para tablet (maestro-detalle)
drawable/	Iconos vectoriales y placeholders
anim/	Animaciones de transición entre fragments
menu/	Menús de navegación, búsqueda y detalle
values/	Strings (ES), colores, temas, arrays de preferencias
values-en/	Strings en inglés (soporte bilingüe)
xml/	Configuración de preferencias y seguridad de red
assets/	recetas.json — fallback offline (~276 KB)

4. Archivos Principales

A continuación se describe la responsabilidad de cada archivo relevante del proyecto.

4.1 Capa de Datos

Archivo	Responsabilidad
RecipeRepository.kt	Fuente única de datos: orquesta API remota → Room → assets con fallback automático.
RecipeDatabase.kt	Definición de la base de datos Room unificada (recetas + favoritos).
RecipeDao.kt	DAO recetas: consultas SQL LIKE para búsquedas por nombre, restaurante e ingredientes.
FavoriteDao.kt	DAO favoritos: operaciones CRUD para la lista de favoritos del usuario.
RecipeApiService.kt	Interfaz Retrofit que declara el endpoint RecetasCocina.json.
RetrofitClient.kt	Singleton Retrofit con Moshi, OkHttp (logging, timeouts 30 s) y FlexibleAdapterFactory.
RecipeResponse.kt	DTOs Moshi con anotaciones @Json y @JsonClass + FlexibleAdapterFactory para JSON irregular.
Mappers.kt	Funciones de mapeo DTO → Entity: parseo de tiempo, URLs de imagen y restaurante.
Recipe.kt	Entidad Room de receta con todos sus campos.
Favorite.kt	Entidad Room de favorito con FK a Recipe.

4.2 Capa de Presentación (UI)

Archivo	Responsabilidad
MainActivity.kt	Activity principal: Navigation Component, BottomNavBar, gestión de Toolbar (hideToolbar/showToolbar).
SplashActivity.kt	Splash Screen compatible con Android 12+ (androidx.core:core-splashscreen).
RecipeListFragment.kt	Lista de recetas: RecyclerView, búsqueda en tiempo real, Chips de filtro y banner offline.
RecipeListViewModel.kt	ViewModel lista: expone LiveData de recetas filtradas y ordenadas por preferencia.
RecipeAdapter.kt	Adapter RecyclerView: bindea imagen (Glide), nombre, restaurante, tiempo y filtro TimeFilter.
RecipeDetailFragment.kt	Detalle de receta: renderiza HTML, FAB favoritos, botón WebView y menú compartir.
RecipeDetailViewModel.kt	ViewModel detalle: gestiona toggle de favorito y exposición del detalle.
FavoritesFragment.kt	Lista de recetas marcadas como favoritas con empty state.
FavoritesViewModel.kt	ViewModel favoritos: observa la tabla favoritos de Room.
SettingsFragment.kt	Preferencias: PreferenceFragmentCompat con tema, intervalo de refresco y orden.
RestaurantWebViewFragment.kt	WebView embebido con WebViewClient personalizado, ProgressBar y manejo de errores.

4.3 Utilidades

Archivo	Responsabilidad
NetworkUtils.kt	Detecta conectividad activa para activar/desactivar el banner de modo offline.
ThemeManager.kt	Gestiona el tema claro/oscuro/sistema aplicando <code>AppCompatActivity.setDefaultNightMode()</code> .
DataLoadingIdlingResource.kt	<code>IdlingResource</code> personalizado para sincronizar Espresso con carga asíncrona de datos.

4.4 Tests

Archivo	Tipo	Cobertura
MappersTest.kt	Unit (JVM)	20 tests: <code>extractRestaurantName</code> , <code>parseTimeToMinutes</code> , <code>extractRestaurantUrl</code> , <code>extractImageUrl</code> .
RecipeListFragmentTest.kt	UI (Barista + Espresso)	5 tests: carga de lista, click en receta, búsqueda, filtro de tiempo, pull-to-refresh.
RecipeDetailFragmentTest.kt	UI (Barista + Espresso)	3 tests: visualización datos, toggle favoritos, botón restaurante.
NavigationTest.kt	UI (Barista + Espresso)	2 tests: navegación <code>BottomNavigation</code> , título toolbar.

5. Librerías y Dependencias

El proyecto hace uso de las siguientes librerías declaradas en `app/build.gradle.kts`:

5.1 Core y Lenguaje

Librería	Versión	Propósito
Kotlin	1.9.22	Lenguaje de programación principal.
Android SDK	26–35	Plataforma mínima (Oreo) y objetivo (Android 15).
Kotlin Coroutines	—	Operaciones asíncronas en ViewModels y Repository.
ViewBinding	—	Acceso tipado y seguro a las vistas del layout.

5.2 Arquitectura y Persistencia

Librería	Versión	Propósito
Room	2.6.1	Base de datos local SQLite: entidades, DAOs y migraciones.
LiveData / ViewModel	—	Componentes de arquitectura MVVM de Jetpack.
Navigation Component	2.7.7	Navegación entre fragments con <code>nav_graph.xml</code> y animaciones.
androidx.preference (Jetpack)	1.2.1	PreferenceFragmentCompat de AndroidX Jetpack, declarado como dependencia en <code>build.gradle.kts</code> .

5.3 Red y Serialización

Librería	Versión	Propósito
Retrofit	2.9.0	Cliente HTTP para consumir la API REST de Turismo Asturias.
OkHttp	4.12.0	Cliente HTTP subyacente: interceptor de logs (BODY) y timeouts de 30 s.
Moshi	1.15.1	Deserialización JSON con soporte Kotlin (null-safety, data classes).
converter-moshi	2.9.0	Adaptador Retrofit–Moshi para parseo automático de respuestas.
moshi-kotlin	1.15.1	KotlinJsonAdapterFactory para delegar parseo de tipos básicos.

5.4 Interfaz de Usuario

Librería	Versión	Propósito
Material Design 3	1.11.0	Sistema de diseño de Google: componentes, temas y colores.
Glide	4.16.0	Carga y caché de imágenes con placeholder y animación crossfade.
SwipeRefreshLayout	1.1.0	Gesto pull-to-refresh en la lista de recetas.
Splash Screen API	1.0.1	Pantalla de bienvenida compatible con Android 12+.

5.5 Testing

Librería	Versión	Propósito
JUnit	4.13.2	Tests unitarios JVM: lógica de parseo en Mappers.
Espresso	3.5.1	Framework base de testing UI Android.
Barista	4.3.0	Wrapper sobre Espresso con API legible (clickOn, assertDisplayed, writeTo), scroll automático y esperas condicionales integradas.
org.json	—	Parseo JSON en tests unitarios JVM sin dependencia de Android.

6. Funcionalidades Implementadas

6.1 Lista de Recetas

- RecyclerView con Cards: imagen, nombre, restaurante y tiempo de preparación.
- Pull-to-refresh para recargar desde la API.
- Chips de filtrado por tiempo: Rápido (≤ 30 min), Medio (31–60 min), Largo (> 60 min).
- Búsqueda en tiempo real por nombre, restaurante e ingredientes (Room SQL LIKE).
- Banner informativo de modo offline cuando no hay conexión.
- Layout adaptativo para tablet (sw600dp): panel maestro-detalle.

6.2 Detalle de Receta

- Imagen grande con Glide + placeholder.
- Nombre, restaurante y tiempo de preparación.
- Ingredientes, preparación y trucos renderizados desde HTML (HtmlCompat).
- FAB para añadir o quitar de favoritos.
- Botón para abrir el restaurante en WebView embebido.
- Menú contextual para compartir la receta (ACTION_SEND).

6.3 Favoritos

- Lista de recetas marcadas como favoritas.
- Persistencia en Room (tabla favorites).
- Empty state con icono y mensaje orientativo.

6.4 Preferencias de Usuario

Clave	Tipo	Valores posibles	Por defecto
refresh_hours	String	1, 6, 12, 24, 48, 168 (horas)	24
default_sort	String	name, restaurant, time	name
hide_no_time	Boolean	true / false	False
Theme	String	Dark, Light, Auto	Auto

6.5 WebView Embebido

- Carga el sitio web del restaurante dentro de la app.
- WebViewClient personalizado: no redirige al navegador externo.
- ProgressBar de carga y manejo de errores de red.
- En modo teléfono, oculta la toolbar principal para evitar barra de título duplicada.

7. Decisiones de Diseño

7.1 Moshi en lugar de Gson

Se utiliza Moshi en lugar de Gson por su mayor integración con Kotlin: soporte nativo de null-safety, mejor rendimiento con data classes y alineación con el ecosistema Square (Retrofit, OkHttp). El FlexibleAdapterFactory resuelve la irregularidad del JSON de la API, donde algunos campos pueden ser objeto o array dinámicamente.

7.2 Estrategia de Fallback

El repositorio implementa un fallback en tres niveles: API remota → caché Room → archivo assets/recetas.json. Esto garantiza que la app funcione sin VPN universitaria y sin conexión a internet, mostrando un banner informativo al usuario.

7.3 Navegación con BottomNavigation

Se eligió BottomNavigation con 3 tabs (Recetas, Favoritos, Ajustes) implementado con Navigation Component, por ser el patrón más adecuado para una app con pocas pantallas principales, siguiendo las guías de Material Design 3.

7.4 Soporte Tablet

El layout alternativo en layout-sw600dp implementa el patrón maestro-detalle: lista seleccionable a la izquierda y detalle de la receta a la derecha en una sola pantalla, aprovechando el espacio extra disponible en tablets.

7.5 Soporte Multidioma

La aplicación soporta español (ES) e inglés (EN) mediante archivos strings.xml separados en values/ y values-en/, siguiendo el mecanismo estándar de recursos de Android.

7.6 Uso de APIs superiores al SDK mínimo 26

Aunque el SDK mínimo declarado es la API 26 (Android 8.0 Oreo), determinadas funcionalidades de la aplicación requieren APIs superiores y se gestionan con comprobaciones de versión en tiempo de ejecución:

- Splash Screen nativa (API 31 / Android 12): La Splash Screen API de AndroidX (core-splashscreen 1.0.1) actúa como capa de compatibilidad. En dispositivos con Android 12+ se delega en el comportamiento nativo del sistema; en versiones anteriores la librería emula el comportamiento de forma transparente, sin necesidad de comprobación manual.
- `ConnectivityManager.getActiveNetwork()` + `getNetworkCapabilities()` (API 23+, cubierto por el minSdk 26 declarado): La detección de conectividad en `NetworkUtils` ya está garantizada por el SDK mínimo.
- Cualquier otra API con nivel superior a 26 se envuelve en bloques `Build.VERSION.SDK_INT >= Build.VERSION_CODES.X` para garantizar compatibilidad descendente y evitar crashes en dispositivos más antiguos.

Esta estrategia permite aprovechar mejoras de la plataforma en dispositivos modernos manteniendo al mismo tiempo la compatibilidad con el 97 % del parque activo de dispositivos Android (API 26+).

7.7 Solución de concurrencia en Ajustes — Handler.post()

Al implementar la pantalla de Ajustes de Jetpack, se detectó un fallo (crash) al cambiar de tema visual. La causa era que el cambio de tema (`AppCompatActivity.setDefaultNightMode()`) recreaba la Activity y destruía el fragmento mientras el callback nativo de la preferencia aún se estaba ejecutando.

Solución: se difirió la aplicación del nuevo tema envolviéndolo en un `Handler(Looper.getMainLooper()).post {}`. Esto permite que la recreación de la interfaz ocurra de manera segura en el siguiente ciclo del bucle de mensajes del sistema, estabilizando por completo la pantalla de ajustes.

7.8 Tests UI con Barista — motivación técnica

Los tests originales con Espresso puro usaban `Thread.sleep()` para esperas, lo que resultaba frágil y ralentizaba la suite. Barista 4.3.0 proporciona:

- API legible: `clickOn(R.id.btn)`, `assertDisplayed(R.id.view)`, `writeTo(R.id.edit, "texto")`.
- Scroll automático antes de interactuar, con soporte nativo de `NestedScrollView`.
- Esperas condicionales integradas en `assertDisplayed()` sin necesidad de `IdlingResource` explícito.
- Coexistencia con Espresso para operaciones avanzadas (`RecyclerViewActions`, matchers complejos).

Resultado: 20 tests unitarios + 10 tests UI pasan correctamente en el dispositivo SM-G990B (Android 14).